



Welcome to QuACK!

Week 2

6/18/2026

Nina, Alyson, & Victoria

Structure of the bootcamp

Session 1 (6/4)	Session 2 (6/18)	Session 3 (7/2)	Session 4 (7/16)
Intro to R and RStudio	Data cleaning and wrangling	Data visualization	Intro to stats with R

We will work with some of the same data throughout the bootcamp to get a feel for the process of exploring data and getting it ready for analysis

Sessions build on each other and we will pick up where we left off in the last session – you will get the most out of the bootcamp if you attend all four weeks!

Today's Agenda

- Warm-up activity
- Dataframes
- Packages
- Intro to Tidyverse
- Useful functions in Tidyverse

Week 2 Warm-up – 10 minutes

1. Download this week's materials from this link: <https://tinyurl.com/quack-summer>
2. Make a New R markdown. Save it to your computer. Title it "S2_warmup"
3. Open RStudio and make sure your working directory is correct.
4. Add a comment with the date, a title, and your name
5. Create three different vectors:
 - a. A vector of your favorite numbers
 - b. A vector of your favorite people
 - c. A vector of your favorite booleans
6. Run the code and check that everything looks correct in the global environment.
7. Open the starter script for this week.



Dataframes are a convenient way to view your data, and they can be configured in a number of different ways to best suit your needs

But there are some near-universal guidelines!

UC Berkeley



Things You Cannot Do

- Multiple data types in one column
- Multiple variables in a column (e.g., date + experimenter initials)
- Info outside of the dataframe (highlights, comments, etc.)
- Spaces in variable names



Data cleaning basics

- Sometimes, cleaning your data will involve changing capitalization, removing whitespace, or changing other details of your data formatting
- Useful functions for this are the `str_detect()` and `grep()` families
- They use RegEx to identify a pattern in your data, which you can then replace with something else

```
# Creating string vector
```

```
x <- c("ALYSON", "NINA")
```

```
# Calling grepl() function
```

```
grepl("N", x)
```

```
# Calling str_detect() function
```

```
str_detect("N", x)
```

Packages

- Base R is powerful on its own, but sometimes you'll need some added functionality
- Packages are collections of functions that other people have written and published for all to use
- R comes with some packages pre-loaded, but some you need to install/load yourself
- Here's how:

```
>> install.packages("packagename")
```

this will install the package and
add it to R on your computer

```
>> library(packagename)
```

this will load the package for
your current session

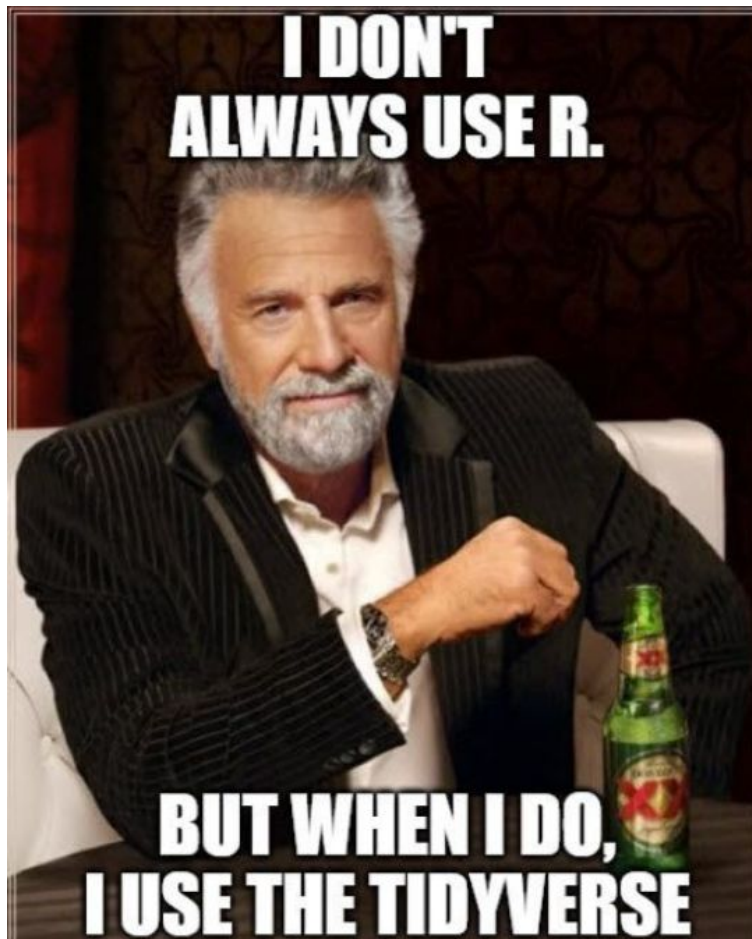


UC Berkeley

Tidyverse

- From their [website](#): “The tidyverse is an opinionated collection of R packages designed for data science. All packages share an underlying design philosophy, grammar, and data structures.”
- Each package has its own focus: reading in data, reshaping data, visualizing data, etc.
- You can install/load them all together using the name “tidyverse” or individually





UC Berkeley

%>%

Introducing: The Pipe Operator

And some useful tidyverse functions

- **select()** for (you guessed it) selecting things like columns that you want to include or drop from a dataframe
- **filter()** for including/removing rows with particular values
- **distinct()** for removing duplicate values
- **mutate()** for changing or adding columns with particular values (and much more)
- **case_when()** for using conditionals with mutate()
- **summarize()** for creating compact dataframes with summary statistics
- **group_by()** for collapsing your data by particular variables (usually for the purpose of summarizing)

Break

UC Berkeley



Sometimes your data is just not quite right...

Maybe it needs to change format or maybe you need to add more rows or columns



And so...



Wide vs. Long Data

Wide Format

Team	Points	Assists	Rebounds
A	88	12	22
B	91	17	28
C	99	24	30
D	94	28	31

Long Format

Team	Variable	Value
A	Points	88
A	Assists	12
A	Rebounds	22
B	Points	91
B	Assists	17
B	Rebounds	28
C	Points	99
C	Assists	24
C	Rebounds	30
D	Points	94
D	Assists	28
D	Rebounds	31

Pivoting / Reshaping Datasets in R

table4a

country	1999	2000
A	0.7K	2K
B	37K	80K
C	212K	213K



country	year	cases
A	1999	0.7K
B	1999	37K
C	1999	212K
A	2000	2K
B	2000	80K
C	2000	213K

pivot_longer(data, cols, names_to = "name",
values_to = "value", values_drop_na = FALSE)

"Lengthen" data by collapsing several columns
into two. Column names move to a new
names_to column and values to a new values_to
column.

```
pivot_longer(table4a, cols = 2:3, names_to = "year",  
values_to = "cases")
```

table2

country	year	type	count
A	1999	cases	0.7K
A	1999	pop	19M
A	2000	cases	2K
A	2000	pop	20M
B	1999	cases	37K
B	1999	pop	172M
B	2000	cases	80K
B	2000	pop	174M
C	1999	cases	212K
C	1999	pop	1T
C	2000	cases	213K
C	2000	pop	1T



country	year	cases	pop
A	1999	0.7K	19M
A	2000	2K	20M
B	1999	37K	172M
B	2000	80K	174M
C	1999	212K	1T
C	2000	213K	1T

pivot_wider(data, names_from = "name",
values_from = "value")

The inverse of pivot_longer(). "Widen" data by
expanding two columns into several. One column
provides the new column names, the other the
values.

```
pivot_wider(table2, names_from = type,  
values_from = count)
```

Ways to Combine Datasets in R

Combine Data Sets

a		b	
x1	x2	x1	x3
A	1	A	T
B	2	B	F
C	3	D	T

+

=

Mutating Joins

x1	x2	x3
A	1	T
B	2	F
C	3	NA

dplyr::left_join(a, b, by = "x1")

Join matching rows from b to a.

x1	x3	x2
A	T	1
B	F	2
D	T	NA

dplyr::right_join(a, b, by = "x1")

Join matching rows from a to b.

x1	x2	x3
A	1	T
B	2	F

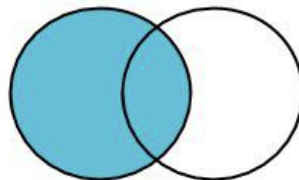
dplyr::inner_join(a, b, by = "x1")

Join data. Retain only rows in both sets.

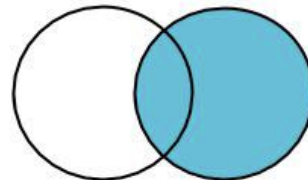
x1	x2	x3
A	1	T
B	2	F
C	3	NA
D	NA	T

dplyr::full_join(a, b, by = "x1")

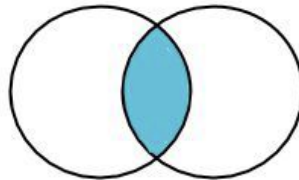
Join data. Retain all values, all rows.



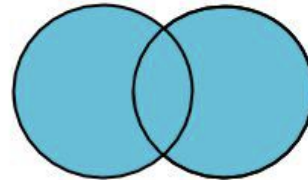
Left Join



Right Join



Inner Join



**Full Outer
Join**

Functions in R to Combine Data

Combine Data Sets

a		b	
x1	x2	x1	x3
A	1	A	T
B	2	B	F
C	3	D	T

+

=

Mutating Joins

x1	x2	x3
A	1	T
B	2	F
C	3	NA

dplyr::left_join(a, b, by = "x1")

Join matching rows from b to a.

x1	x3	x2
A	T	1
B	F	2
D	T	NA

dplyr::right_join(a, b, by = "x1")

Join matching rows from a to b.

x1	x2	x3
A	1	T
B	2	F

dplyr::inner_join(a, b, by = "x1")

Join data. Retain only rows in both sets.

x1	x2	x3
A	1	T
B	2	F
C	3	NA
D	NA	T

dplyr::full_join(a, b, by = "x1")

Join data. Retain all values, all rows.

Left Join

```
joined_df <- left_join(a,b)
```

```
joined_df <- a %>%  
  left_join(b)
```

Full Join

```
joined_df <- full_join(a,b)
```

```
joined_df <- a %>%  
  full_join(b)
```

Inner Join

```
joined_df <- inner_join(a,b)
```

```
joined_df <- a %>%  
  inner_join(b)
```

An Example of When We Might Use These

- Let's pretend we have a new dataset from a survey collecting Mid-Semester feedback on Quack.
- We might want to compare these responses to our Beginning-of-Semester Quack survey responses.
- In order to do this, we will need to first combine both datasets together in R.
- Let's take a look at some use-cases for each type of joining using these surveys.

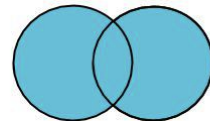
Example: Full Join

a		b		
x1	x2	x1	x3	
A	1	A	T	
B	2	B	F	
C	3	D	T	

+ =

x1	x2	x3
A	1	T
B	2	F
C	3	NA
D	NA	T

`dplyr::full_join(a, b, by = "x1")`
Join data. Retain all values, all rows.



Full Outer
Join

- Maybe we want to look at all the data we have collected this semester and don't care if people filled out both or only one survey.

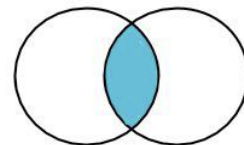
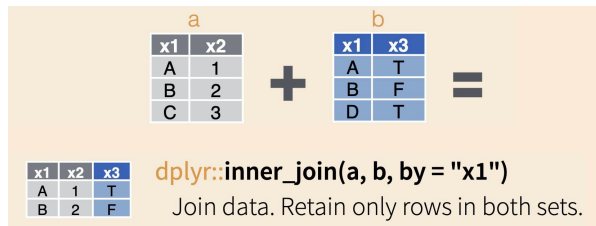
```
full_join(Beginning-of-Semester Data, Mid-Semester Data, by = "ID")
```



Ordering doesn't matter since all rows in both dataframes are retained

→ *Result: dataset with everyone from both datasets, regardless of whether they completed both or only one of the survey*

Example: Inner Join



Inner Join

- If we want to see whether ratings improved over the course of the semester so far, we probably want to look *only* at people who have completed both surveys.

```
inner_join(Beginning-of-Semester Data, Mid-Semester Data, by = "ID")
```



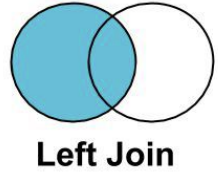
Ordering doesn't matter since only rows that exist in both datasets will be retained

→ Result: dataset with ONLY people who completed BOTH surveys

Example: Left Join

a		b		
x1	x2	x1	x3	
A	1	A	T	+
B	2	B	F	
C	3	D	T	
				=
x1	x2	x3		
A	1	T		
B	2	F		
C	3	NA		

dplyr::left_join(a, b, by = "x1")
Join matching rows from b to a.



- Let's say we only want to look at people who filled out the Beginning-of-Semester survey.
- Maybe we don't care if they also completed the Mid-Semester survey, but if they did, we want to incorporate that data too.

```
left_join(Beginning-of-Semester Data, Mid-Semester Data, by = "ID")
```

↑
All rows in this data will be retained

↑
Rows that do not also exist in Beginning-of-Semester data will be removed

→ *Result: dataset with only people who completed Beginning-of-Semester survey + adding their corresponding Mid-Semester if applicable*

And lastly, a cheat sheet!

<https://rstudio.github.io/cheatsheets/html/data-transformation.html>

dplyr functions work with pipes and expect **tidy data**. In tidy data:



Each **variable** is in its own column



Each **observation**, or **case**, is in its own row

pipes

x |> **f(y)** becomes **f(x, y)**

Summarize Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

summary function

```
summarize(data, ...)
  Compute table of summaries.
  mtcars |> summarize(avg = mean(mpg))

count(data, ..., wt = NULL, sort = FALSE, name = NULL)
  Count number of rows in each group defined by the variables in ... Also tally(), add_count(), add_tally().
  mtcars |> count(cyl)
```

Group Cases

Use **group_by(data, ..., add = FALSE, drop = TRUE)** to create a "grouped" copy of a table grouped by columns in ... dplyr functions will manipulate each "group" separately and combine the results.

```
mtcars |> group_by(cyl) |> summarize(avg = mean(mpg))
```

Use **rowwise(data, ...)** to group data into individual rows. dplyr functions will compute results for each row. Also apply functions to list-columns. See tidy cheat sheet for list-column workflow.

```
starwars |> rowwise() |> mutate(film_count = length(films))
```

ungroup(x, ...) Returns ungrouped copy of table.
g_mtcars <- mtcars |> group_by(cyl)
ungroup(g_mtcars)

Manipulate Cases

EXTRACT CASES

Row functions return a subset of rows as a new table.

```
filter(data, ..., preserve = FALSE)
  Extract rows that meet logical criteria.
  mtcars |> filter(mpg > 20)

distinct(data, ..., keep_all = FALSE)
  Remove rows with duplicate values.
  mtcars |> distinct(gear)

slice(data, ..., preserve = FALSE)
  Select rows by position.
  mtcars |> slice(10:15)

slice_sample(data, ..., n, prop, weight_by = NULL, replace = FALSE)
  Randomly select rows. Use n to select a number of rows and prop to select a fraction of rows.
  mtcars |> slice_sample(n = 5, replace = TRUE)

slice_min(data, order_by, ..., n, prop, with_ties = TRUE) and slice_max(data, order_by, ..., n, prop, with_ties = TRUE)
  Select rows with the lowest and highest values.
  mtcars |> slice_min(mpg, prop = 0.25)

slice_head(data, ..., n, prop) and slice_tail(data, ..., n, prop)
  Select the first or last rows.
  mtcars |> slice_head(n = 5)
```

Logical and boolean operators to use with filter()

==	<	<=	is.na()	%in%		xor()
!=	>	>=	!is.na()	!	&	

See ?base::Logic and ?Comparison for help.

ARRANGE CASES

```
arrange(data, ..., by_group = FALSE)
  Order rows by values of a column or columns (low to high), use with desc() to order from high to low.
  mtcars |> arrange(mpg)
  mtcars |> arrange(desc(mpg))
```

ADD CASES

```
add_row(data, ..., before = NULL, after = NULL)
  Add one or more rows to a table.
  cars |> add_row(speed = 1, dist = 1)
```

Summary Functions

TO USE WITH SUMMARIZE()

summarize() applies summary functions to columns to create a new table. Summary functions take vectors as input and return single values as output.

summary function

COUNT

```
dplyr::n() - number of values/rows
dplyr::n_distinct() - # of uniques
sum(is.na()) - # of non-NA's
```

POSITION

```
mean() - mean, also mean(is.na())
median() - median
```

LOGICAL

```
mean() - proportion of TRUEs
sum() - # of TRUEs
```

ORDER

```
dplyr::first() - first value
dplyr::last() - last value
dplyr::nth() - value in nth location of vector
```

RANK

```
quantile() - nth quantile
min() - minimum value
max() - maximum value
```

SPREAD

```
IQR() - Inter-Quartile Range
mad() - median absolute deviation
sd() - standard deviation
var() - variance
```

Row Names

Tidy data does not use rownames, which store a variable outside of the columns. To work with the rownames, first move them into a column.

```
tibble::rownames_to_column()
  Move row names into col.
  a <- mtcars |> rownames_to_column(var = "C")
```

```
tibble::column_to_rownames()
  Move col into row names.
  a |> column_to_rownames(var = "C")
```

Also **tibble::has_rownames()** and **tibble::remove_rownames()**.

Combine Tables

COMBINE VARIABLES

```
x + y
  Returns tables side by side as a single table. Column lengths must be equal. Columns will NOT be matched by id (to do that look at Relational Data below), so be sure to check that both tables are ordered the way you want before binding.

bind_cols(..., name_repair)
  Returns tables placed side by side as a single table. Column lengths must be equal. Columns will NOT be matched by id (to do that look at Relational Data below), so be sure to check that both tables are ordered the way you want before binding.
```

RELATIONAL DATA

Use a "Mutating Join" to join one table to columns from another, matching values with the rows that they correspond to. Each join retains a different combination of values from the tables.

```
left_join(x, y, by = NULL, copy = FALSE, suffix = c("x", "y"), ..., keep = FALSE, na_matches = "na")
  Join matching values from x to y.
```

```
right_join(x, y, by = NULL, copy = FALSE, suffix = c("x", "y"), ..., keep = FALSE, na_matches = "na")
  Join matching values from y to x.
```

```
inner_join(x, y, by = NULL, copy = FALSE, suffix = c("x", "y"), ..., keep = FALSE, na_matches = "na")
  Join data. Retain only rows with matches.
```

```
full_join(x, y, by = NULL, copy = FALSE, suffix = c("x", "y"), ..., keep = FALSE, na_matches = "na")
  Join data. Retain all values, all rows.
```

COLUMN MATCHING FOR JOINS

```
Use by = c("col1", "col2", ...) to specify one or more common columns to match on.
left_join(x, y, by = "A")
```

```
Use a named vector, by = c("C" = "I" = "col2") = "col2", to match on columns that have different names in each table.
left_join(x, y, by = c("C" = "D"))
```

```
Use suffix to specify the suffix to give to unmatched columns that have the same name in both tables.
left_join(x, y, by = c("C" = "D"), suffix = c("1", "2"))
```

COMBINE CASES

```
x
+ y
  bind_rows(..., id = NULL)
  Returns tables one on top of the other as a single table. Set id to a column name to add a column of the original table names (as pictured).
```

Use a "Filtering Join" to filter one table against the rows of another.

```
x
+ y
  semi_join(x, y, by = NULL, copy = FALSE, ..., na_matches = "na")
  Return rows of x that have a match in y. Use to see what will be included in a join.
```

```
anti_join(x, y, by = NULL, copy = FALSE, ..., na_matches = "na")
  Return rows of x that do not have a match in y. Use to see what will not be included in a join.
```

Use a "Nest Join" to inner join one table to another into a nested data frame.

```
nest_join(x, y, by = NULL, copy = FALSE, keep = FALSE, name = NULL, ...)
  Join data, nesting matches from y in a single new data frame column.
```

SET OPERATIONS

```
intersect(x, y, ...)
  Rows that appear in both x and y.
```

```
setdiff(x, y, ...)
  Rows that appear in x but not y.
```

```
union(x, y, ...)
  Rows that appear in x or y, duplicates removed. union_all() retains duplicates.
```

Use **setequal()** to test whether two data sets contain the exact same rows (in any order).



Exit Ticket

Link: <https://tinyurl.com/quack-summer2026-exitsurvey>

